15: C/Linux Interfacing II

Engr 315: Hardware / Software Codesign

Andrew Lukefahr
Indiana University

Announcements

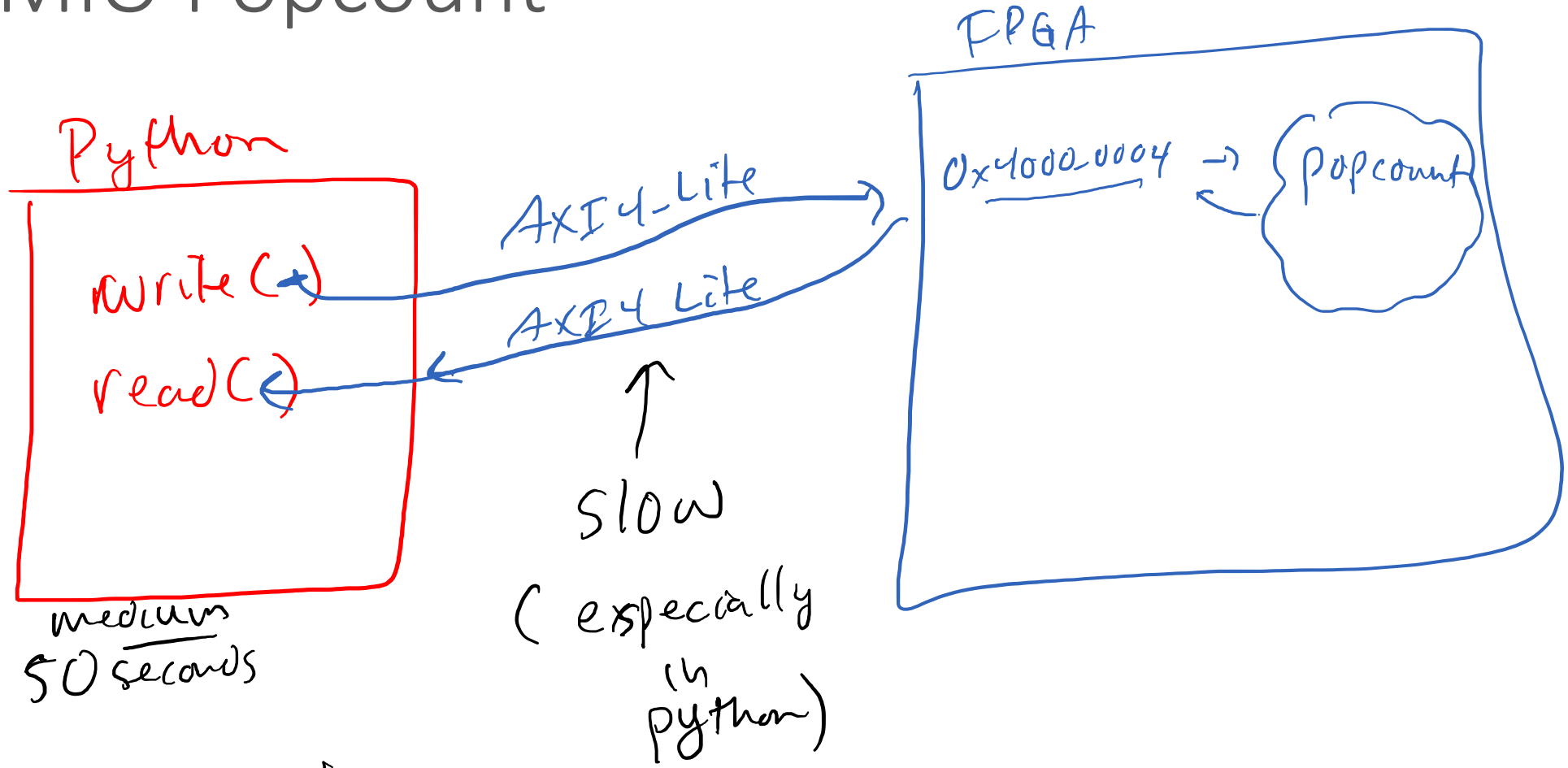
- P5 is out
  - Adds popcount DMA interface

Verilog: AXI-4-Stream interface for Popcount

Python: DMA interface to drive AXI-4 stream

- Exam grading this week
- P6: Dot product in C on Linux

P4: MMIO Popcount



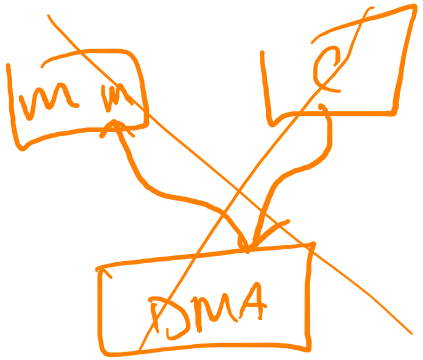
medium

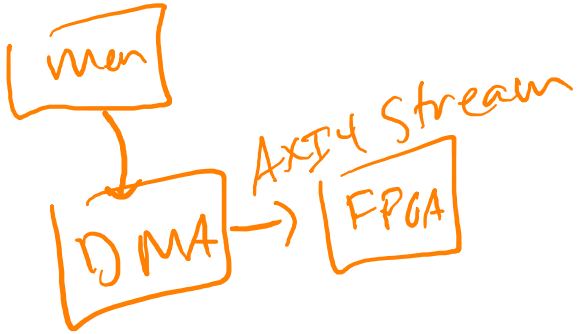
~ python: 50 seconds

MMIO: ~7 seconds

PS: DMA: <1 second

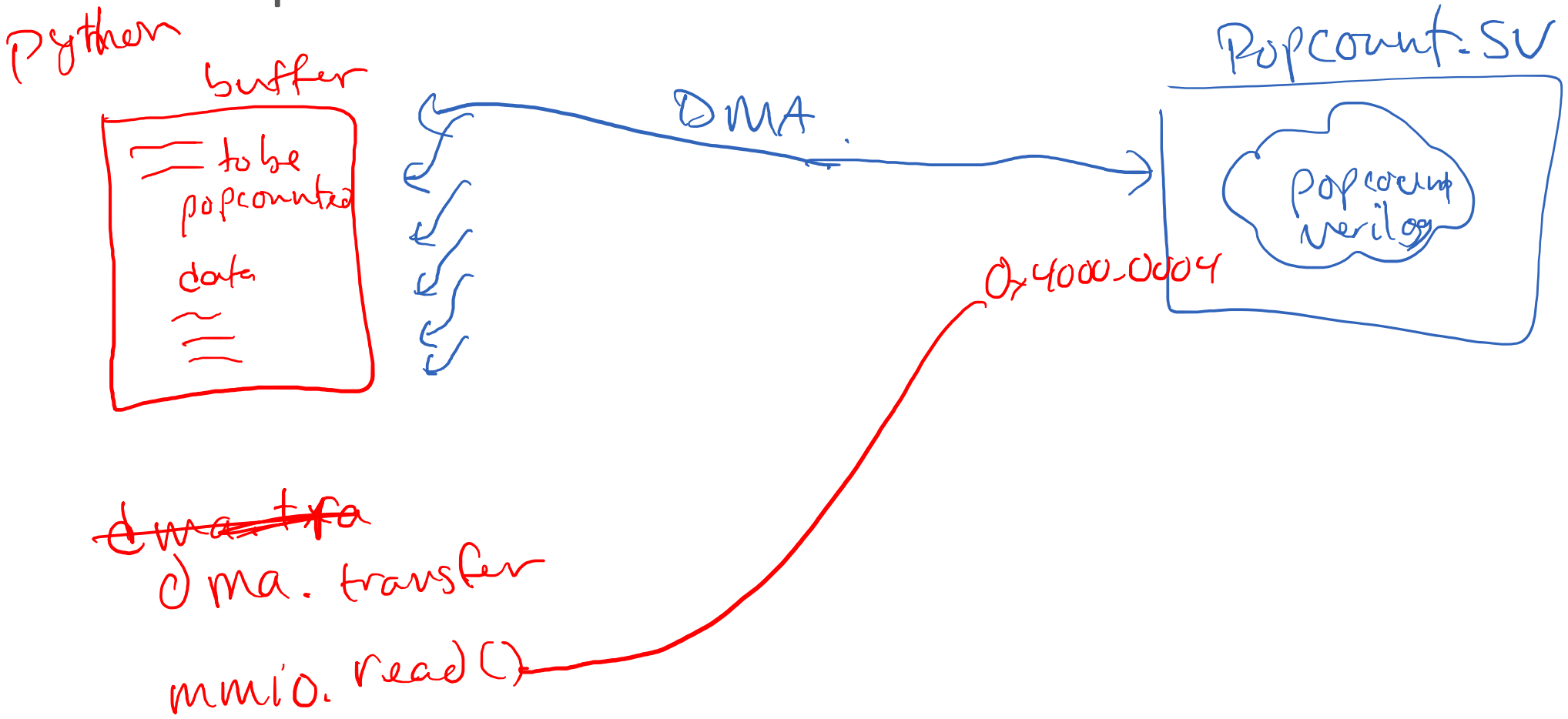
P5: Adds DMA + AXI-Stream to Popcount

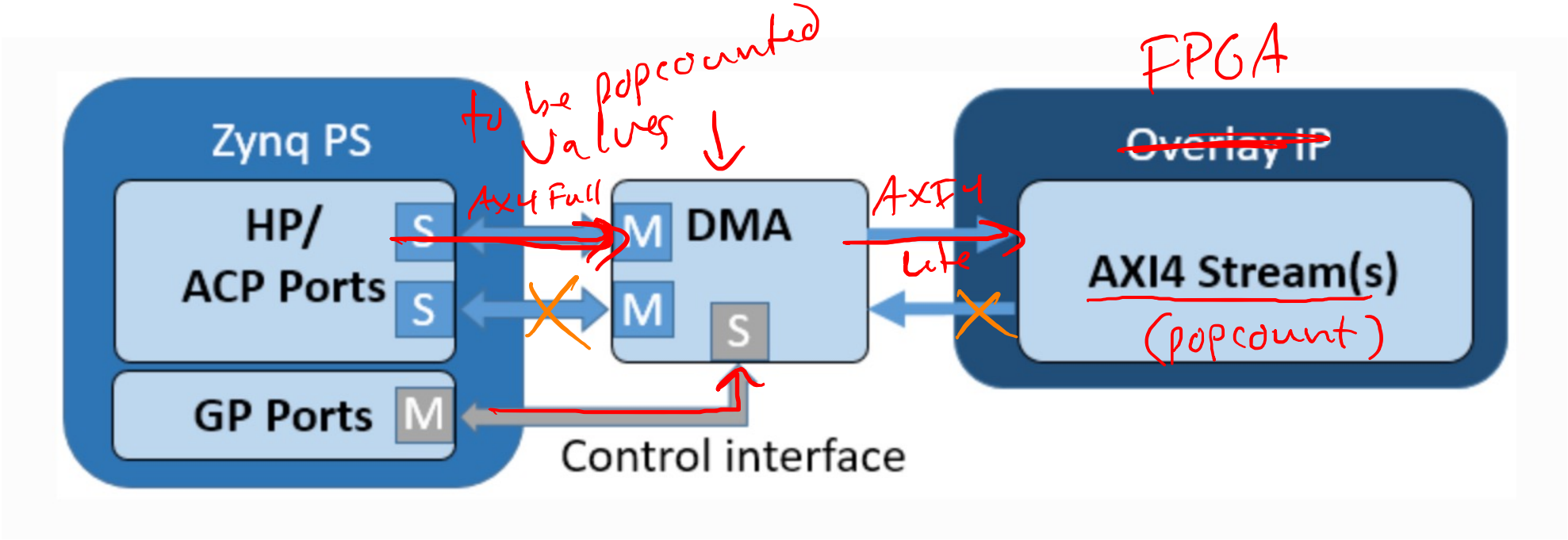




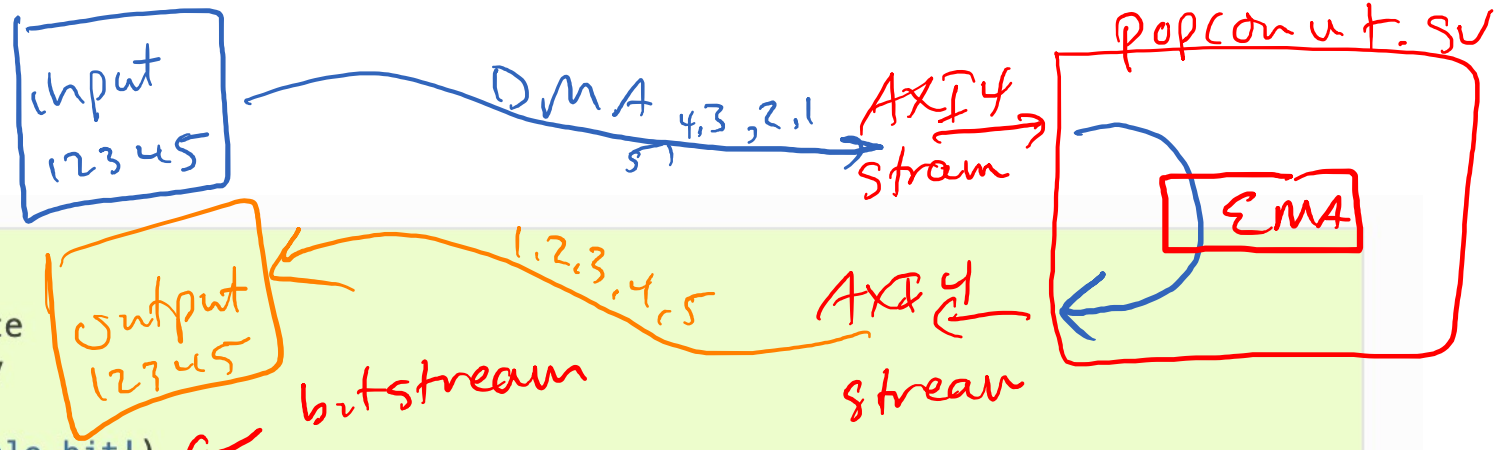
- DMA
  - Add DMA engine to move data via AXI4-Full to AXI-Stream interface
- Popcount.sv:
  - Add AXI-Stream Interface
  - Keep AXI4-Lite Interface to read result

Ideal Popcount DMA





https://pynq.readthedocs.io/en/v2.6.1/pynq_libraries/dma.html




```python
import numpy as np
from pynq import allocate
from pynq import Overlay

overlay = Overlay('example.bit')
dma = overlay.axi_dma

input_buffer = allocate(shape=(5,), dtype=np.uint32)
output_buffer = allocate(shape=(5,), dtype=np.uint32)
```

```python
for i in range(5):
    input_buffer[i] = i
```

1, 2, 3, 4, 5

```python
dma.sendchannel.transfer(input_buffer)
dma.recvchannel.transfer(output_buffer)
dma.sendchannel.wait()
dma.recvchannel.wait()
```

wait until DMA Transfer is done

```
Output buffer will contain: [0 1 2 3 4]
```

https://pynq.readthedocs.io/en/v2.6.1/pynq_libraries/dma.html

P3's DMA with Python

```
def ema_1x(self, data):
    #convert to unsigned int
    data = np.array(data, dtype=np.uint32)

    input_buffer = pynq.allocate(shape=(len(y_sum),), dtype=np.uint32)
    output_buffer = pynq.allocate(shape=(len(y_sum),), dtype=np.uint32)

    # copy into our input_buffer
    np.copyto(input_buffer, data)

    for s in np.arange(0,len(y_sum), self.buf_max):
        e = min(s + self.buf_max, len(y_sum))

        self.ema_1_dma.sendchannel.transfer(input_buffer[s:e])
        self.ema_1_dma.recvchannel.transfer(output_buffer[s:e])
        self.ema_1_dma.sendchannel.wait()
        self.ema_1_dma.recvchannel.wait()

    return output_buffer
```

FPGA

FPGA

So how do we access the FPGA from Linux?

- FPGA uses physical address
- CPU (w/Linux) uses virtual address

/dev/mem

- */dev/mem* is a character device file that is an image of the main memory of the computer. It may be used, for example, to examine (and even patch) the system.
- Byte addresses in */dev/mem* are interpreted as physical memory addresses. References to nonexistent locations cause errors to be returned.
- Requires root (sudo) access

Userspace I/O

- Linux provides a more restrictive version of /dev/mem called /dev/uio
- Linux exposes a small portion of I/O memory space to the user through /dev/uioX interfaces.
- Treat it just like /dev/mem, but not the entire memory space

mmap – memory map

- **mmap**() creates a new mapping in the virtual address space of the calling process. The starting address for the new mapping is specified in *addr*.

- The contents of a file mapping are initialized using *length* bytes starting at offset *offset* in the file (or other object) referred to by the file descriptor *fd*.

python 1st → then C keep in python → Python: easier (x2)

Userspace Popcount w/C

C: learned more (?)

```
#include <fcntl.h>
#include <stdlib.h>
#include <stdint.h>
#include <stdio.h>
#include <sys/mman.h>
#include <unistd.h>

int main (){

    int dev_mem_fd = -1;
    void * vaddr_base;

    //Mapping user-space I/O
    dev_mem_fd = open("/dev/uio0", O_RDWR|O_SYNC);
    if (dev_mem_fd < 0) { perror("open() /dev/uio0"); return 1; }

    // Map 1KB of physical memory starting at 0x40000000
    // to 1KB of virtual memory starting at vaddr_base
    vaddr_base = mmap(0, 1024, PROT_READ|PROT_WRITE,
        MAP_SHARED, dev_mem_fd, 0x0); // not 0x40000000

    if (vaddr_base == MAP_FAILED) { perror("mmap()"); return 1; }

    uint32_t * reset = (uint32_t*) vaddr_base;
    uint32_t * count = (uint32_t*) (vaddr_base + 0x4);

    *reset = 1; //reset count
    *count = 0xf; //add new value
    printf("Count: %d\n", *count); //read count

    if (munmap(vaddr_base, 1024) != 0) { perror("munmap()"); }
    if (close(dev_mem_fd) != 0) { perror("close()"); }
    dev_mem_fd = -1;

    return 0;
}
```

0x4000_0000

0x1234_abcd

loop

MMIO Popcount

PS: DMA in Python

→ DMA in C(?)

- Python

```
Timing 'medium.bin'
Found 4196661 Ones
Total Time:7.867364578007255 seconds
```

7.80 ← Trey Crank up clock speed! :)

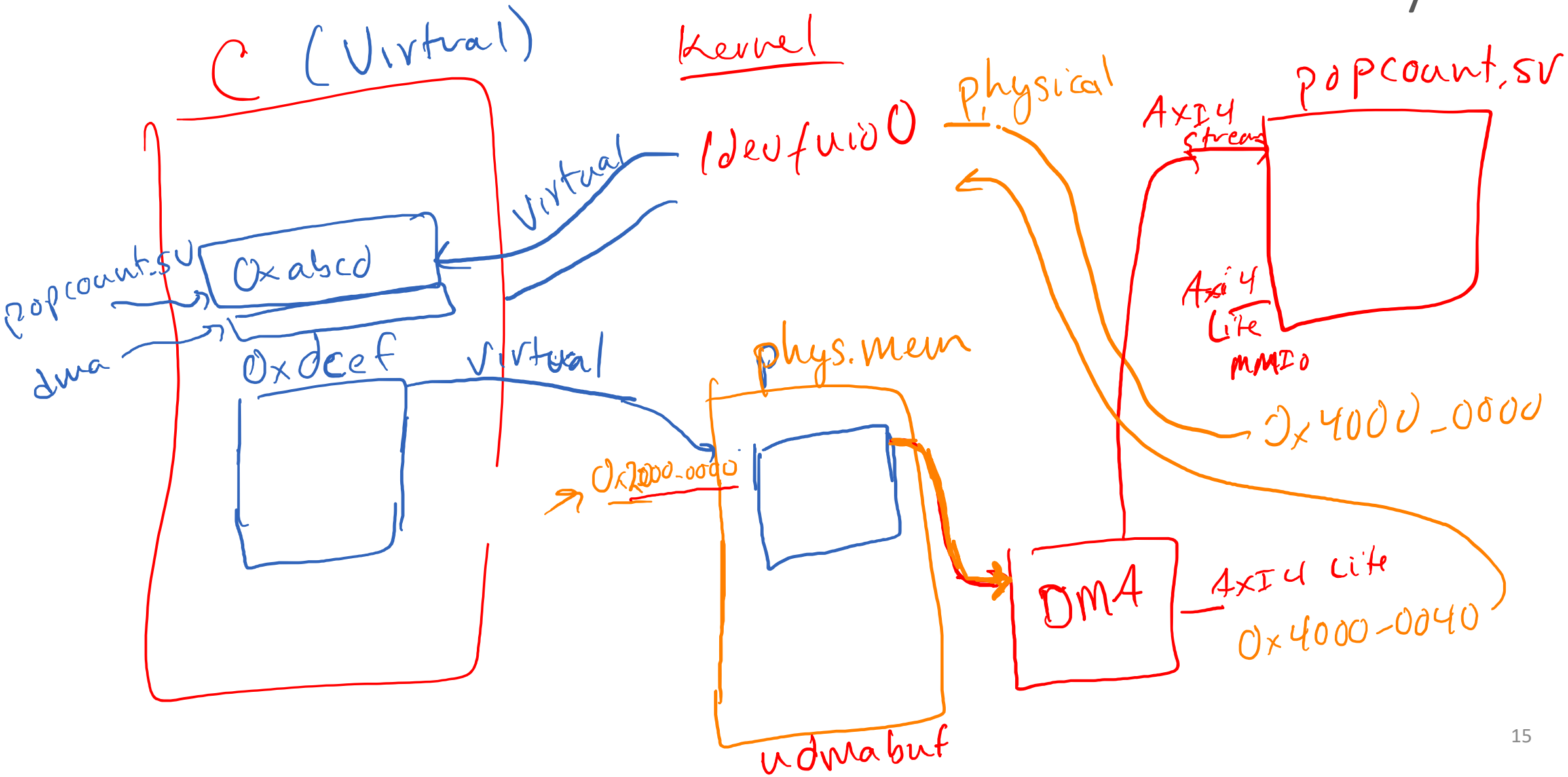
- C

$ time sudo ./popcount_mmio_devmem medium.bin

Counted 4196661 ones

real 0m0.143s

Problem: DMA buffers and Virtual Memory?



Solution: udmabuf

- Kernel module that allows users access to a DMA-able buffer
- User-space contagious DMA buffer with known physical address
- Let’s compile and build it…

Building / Installing udmabuf

- Kernel steps:
  - `cd /lib/modules/5.4.0-xilinx-v2020.1/build`
  - `sudo make scripts prepare`

- `git clone https://github.com/ikwzm/udmabuf`
- `cd udmabuf`
- `Make`
- `sudo insmod u-dma-buf.ko udmabuf0=1048576`

```
xilinx@pynq:~/udmabuf$ cat /sys/class/u-dma-buf/udmabuf0/phys_addr
0x16900000
```

Problem 2: DMA control?

AXI DMA Register Address Map

Table 2-5: **Scatter / Gather Mode Register Address Map**

Address Space Offset[1]	Name	Description
00h	MM2S_DMACR	MM2S DMA Control register
04h	MM2S_DMASR	MM2S DMA Status register
08h	MM2S_CURDESC	MM2S Current Descriptor Pointer. Lower 32 bits of the address.
0Ch	MM2S_CURDESC_MSB	MM2S Current Descriptor Pointer. Upper 32 bits of address.
10h	MM2S_TAILDESC	MM2S Tail Descriptor Pointer. Lower 32 bits.
14h	MM2S_TAILDESC_MSB	MM2S Tail Descriptor Pointer. Upper 32 bits of address.
2Ch [2]	SG_CTL	Scatter/Gather User and Cache
30h	S2MM_DMACR	S2MM DMA Control register
34h	S2MM_DMASR	S2MM DMA Status register
38h	S2MM_CURDESC	S2MM Current Descriptor Pointer. Lower 32 address bits
3Ch	S2MM_CURDESC_MSB	S2MM Current Descriptor Pointer. Upper 32 address bits.
40h	S2MM_TAILDESC	S2MM Tail Descriptor Pointer. Lower 32 address bits.
44h	S2MM_TAILDESC_MSB	S2MM Tail Descriptor Pointer. Upper 32 address bits.

DMA control steps:

Next Time:

- DMA interfacing with C

15: C/Linux Interfacing II

Engr 315: Hardware / Software Codesign

Andrew Lukefahr
Indiana University